

Appendix (Multi-Agent Dual-Path Verification Framework for Scalable Graph Reasoning)

A Prompt Templates

A.1 Router Agent

Prompt Template

```
system: "You are a query router for a hybrid neural-
symbolic graph reasoning system."
user_template: |
  Query: {query}
  TaskType: {task_type}
  GraphStats: {graph_stats}

Decide route:
- symbolic: pick this when the query asks for a
  computation that has a precise procedural answer
  (path, flow, matching, substructure, traversal,
  counting). The system will generate and
  execute code.
- neural: pick this when the query asks for an
  interpretation, preference, or attribute-based
  judgment that does not reduce to a single
  procedure. The system will reason directly.

Output JSON:
{{
  "route": "symbolic" or "neural",
  "why": "Brief reasoning",
  "need_decomposition": true/false,
  "decomposition": {{
    "seed_nodes": [],
    "hop": {default_hop},
    "node_budget": {node_budget},
    "edge_budget": {edge_budget}
  }}
}}
```

A.2 Planner Agent

Prompt Template

```
system: "You are a graph task planner. Analyze the
problem and outline logical steps for an
implementation."
user_template: |
  Task: {query}
  TaskType: {task_type}
  GraphStats: {graph_stats}
  GraphDataHint: {graph_hint}

Provide a high-level execution plan:
- algorithm_family: the category of algorithm (e.g
  ., BFS, flow, backtracking, isomorphism).
- key_steps: logical steps to solve the problem
  programmatically.
- defensive_checks: edge cases to handle (
```

```
disconnected graph, missing nodes, type
mismatch).
```

```
Output JSON:
{{
  "algorithm_family": "...",
  "key_steps": [...],
  "defensive_checks": [...]}
}}
```

A.3 Coder Agent

Prompt Template

```
system: "You are an expert Python programmer
specializing in NetworkX. Write robust, fault-
tolerant code."
user_template: |
  Task: {query}
  TaskType: {task_type}
  Plan: {plan_json}

Graph Input Data:
{graph_payload_hint}

Constraints:
1. Use only standard Python libraries and networkx.
2. Do not install new packages.
3. Print only the final answer. Do not print debug
  info.
4. Wrap dangerous code in try/except blocks and
  print a safe fallback if it fails.

If this is a retry, fix the error:
{error_feedback}

Write the complete Python code in a single ``
python`` block.
```

A.4 Reasoner Agent

Prompt Template

```
system: "You are a logical graph reasoner. Solve the
problem step-by-step using semantic reasoning
."
user_template: |
  Task: {query}
  TaskType: {task_type}
  GraphSummary: {graph_hint}

Perform chain-of-thought reasoning and provide the
final answer.
```

Answer:

A.5 Critic Agent

Prompt Template

```
system: "You are a strict execution verifier.
Compare two outputs for logical equivalence."
consistency_template: |
Query: {query}
OutputA: {out_a}
OutputB: {out_b}

Compare the two outputs.
1. If both are effectively the same (e.g., "Yes"
vs "True", "0-1" vs "1-0"), output consistent=
true.
2. If one output implies failure (e.g., "No", "
Error", "No path") and the other provides a
specific concrete solution, prefer the concrete
solution unless it is obviously hallucinated.
3. If they are logically contradictory (e.g., "
Node 1" vs "Node 2"), explain the conflict.

Output JSON:
{{
  "consistent": true/false,
  "why": "Brief explanation",
  "resolution": "pick_a" or "pick_b" or "
need_tiebreaker",
  "tiebreaker_hint": "Explain what specific check
is needed to verify the truth."
}}
```

B Per-Benchmark Detailed Results

B.1 GraphWiz

Table 1: Performance comparison on GraphWiz benchmark in terms of accuracy (%).

Task	GPT	GraphTeam	DuVerG
Cycle	82.0	100.0	99.2
Connect	64.8	86.5	100.0
Bipartite	67.3	96.3	100.0
Topology	100.0	100.0	95.0
Shortest	8.5	95.0	100.0
Triangle	10.3	94.0	89.2
Flow	19.3	94.0	97.5
Hamilton	36.3	32.5	99.2
Subgraph	40.3	99.3	100.0
Avg	47.6	88.6	97.8

B.2 NLGraph

Table 2: Performance comparison on NLGraph benchmark in terms of accuracy (%).

Task	GPT	GraphTeam	DuVerG
Connect	72.0	97.0	100.0
Cycle	52.9	100.0	100.0
Topology	20.0	94.8	97.8
SP	35.9	98.4	100.0
MaxFlow	98.3	100.0	98.3
Bipartite	29.8	100.0	98.8
Hamilton	22.4	100.0	100.0
GNN	0	97.4	100.0
Avg	51.4	97.8	99.5

B.3 Talk Like a Graph

Table 3: Performance comparison on Talk Like a Graph benchmark in terms of accuracy (%).

Task	GPT	GraphTeam	DuVerG
NCount	100.0	100.0	99.3
ECount	45.6	99.2	100.0
EExist	87.0	61.0	100.0
NDeg	54.6	98.6	99.3
CNodes	50.2	99.2	100.0
Cycle	92.6	99.4	100.0
SP	45.6	99.1	100.0
TriCount	14.4	99.2	99.3
DiscNodes	41.3	54.0	84.7
MaxFlow	36.7	77.3	98.0
NClass	51.3	74.0	100.0
Reach	97.3	98.0	100.0
Avg	59.7	88.3	98.4

B.4 GraphInstruct

Table 4: Performance comparison on GraphInstruct benchmark in terms of accuracy (%).

Task	GPT	GraphTeam	DuVerG
Neighbor	98.0	100.0	99.0
Bipartite	89.7	99.0	100.0
Edge	90.7	99.3	94.7
Connectivity	92.2	100.0	99.7
Degree	98.0	100.0	99.7
DFS	89.3	95.7	97.0
Predecessor	47.4	98.0	99.7
TopoSort	59.0	98.3	99.7
ConnComp	39.3	94.7	99.7
CommonNbr	49.0	100.0	99.0
Hamilton	33.7	97.7	100.0
Jaccard	17.0	100.0	99.0
SP	57.3	100.0	100.0
Diameter	23.0	100.0	99.7
PageRank	15.3	16.3	91.7
MaxFlow	15.0	99.3	100.0
MST	1.0	0	100.0
Avg	53.7	88.1	98.7

B.5 GraphArena

Table 5: Performance comparison on GraphArena benchmark in terms of accuracy (%).

Task	GPT	GraphTeam	DuVerG
Connect	56.1	100.0	95.1
Diameter	32.3	0.0	95.0
Distance	61.8	92.4	99.3
GED	20.0	3.1	38.0
MCP	29.6	98.0	99.7
MCS	50.7	12.7	86.0
MIS	26.9	68.3	90.6
MVC	14.7	9.7	49.3
Neighbor	82.0	98.8	99.8
TSP	14.3	23.0	63.7
Avg	39.2	50.6	83.8

B.6 OGB Benchmarks

Table 6: Performance comparison on OGB benchmarks in terms of accuracy (%).

Dataset / Task	GPT	GraphTeam	DuVerG
<i>ogbn-arxiv</i>			
TriCount	13.5	99.2	100.0
ClustCoef	14.6	83.5	100.0
3-Reach	44.2	99.3	98.1
PageRank	15.5	30.3	76.2
Avg	22.4	79.0	93.5
<i>ogbl-ppa</i>			
TriCount	3.2	100.0	100.0
ClustCoef	3.8	88.0	100.0
3-Reach	30.9	85.4	83.8
PageRank	3.2	21.1	47.8
Avg	10.5	74.6	82.6